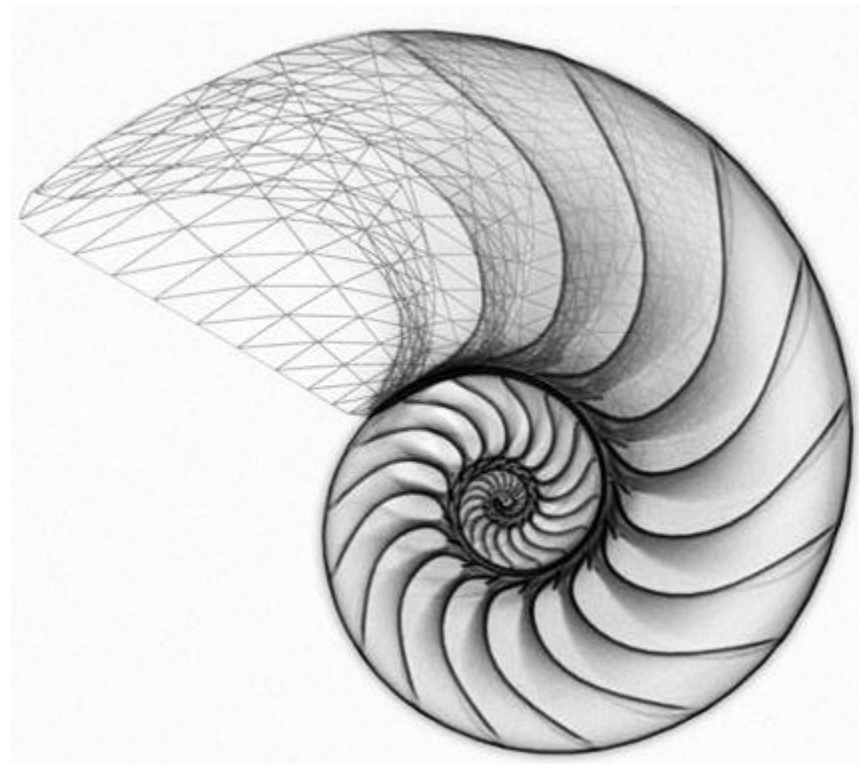




Lecture 05

Analysis of Algorithms

CSE225: Data Structures and Algorithms

Analysis of Algorithms

- An *algorithm* is a finite set of precise instructions for performing a computation or for solving a problem.
 - It must produce the correct result
 - It must finish in some finite time
 - You can represent an algorithm using pseudocode, flowchart, or even actual code

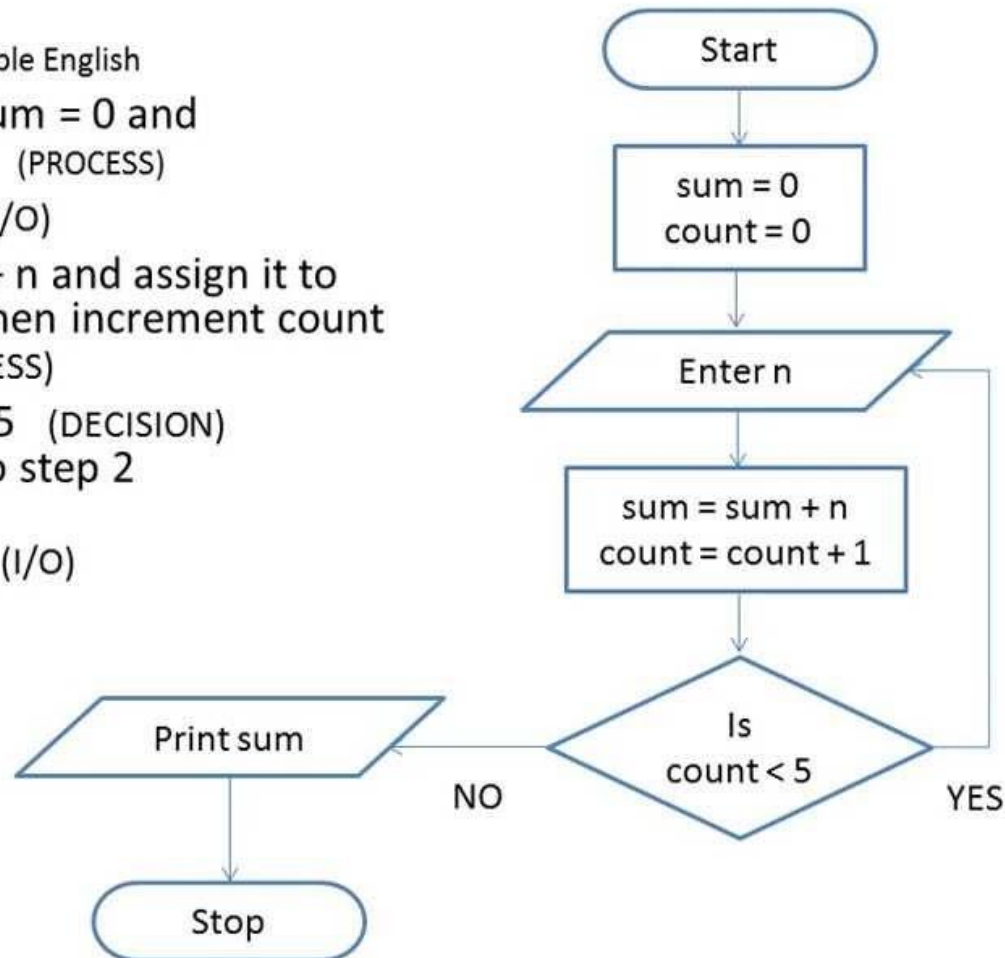
Analysis of Algorithms

Find the sum of 5 numbers

Flowchart

Algorithm in simple English

1. Initialize $\text{sum} = 0$ and $\text{count} = 0$ (PROCESS)
2. Enter n (I/O)
3. Find $\text{sum} + n$ and assign it to sum and then increment count by 1 (PROCESS)
4. Is $\text{count} < 5$ (DECISION)
if YES go to step 2
else
Print sum (I/O)



Analysis of Algorithms

- What does it mean to analyze an algorithm?
 - **To have an estimate about how much time an algorithm may take to finish, or in other words, to analyze its running time**
 - Sometimes, instead of running time, we are interested in how much memory the algorithm may consume while it runs
 - It enables us to compare between two algorithms
- What do we mean by running time analysis?
 - Also referred to as time-complexity analysis
 - To determine how running time increases as the **size of the problem** increases.

Analysis of Algorithms

- Size of the problem can be a range of things, including
 - size of an array
 - polynomial degree of an equation
 - number of elements in a matrix
 - number of bits in the binary representation of the input
 -
 -
- and so on...

How Do We Analyze Running Time?

- We need to define an objective measure.
 - (1) Compare execution times?
 - ***Not good***: times are specific to a particular computer !!



How Do We Analyze Running Time?

- We need to define an objective measure.
 - (2) Count the number of statements executed?
 - Associate a "cost" with each statement.
 - Find the "total cost" by finding the total number of times each statement is executed.
 - **Not good**: number of statements vary with the programming language as well as the style of the individual programmer.

Algorithm 1

	Time
arr[0] = 0;	c_1
arr[1] = 0;	c_1
arr[2] = 0;	c_1
...	
arr[N-1] = 0;	c_1

	$c_1 + c_1 + \dots + c_1 = c_1 N$

Algorithm 2

	Time
for(^{c_1} i=0; ^{c_2} i<N; ^{c_3} i++)	$c_1 + c_2(N+1) + c_3 N$
arr[i] = 0;	$c_1 N$

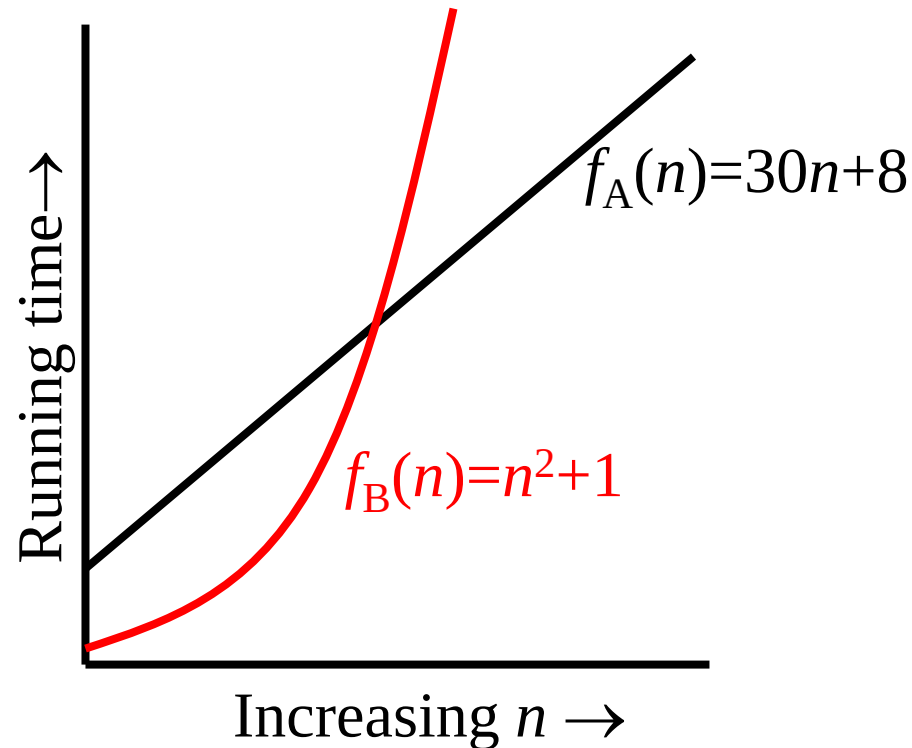
	$c_1 + c_2(N+1) + c_3 N + c_1 N = (c_1 + c_2 + c_3)N + (c_1 + c_2)$

Ideal Solution

- Express running time as a function of the input size n (i.e., $f(n)$).
- Compare different functions of running times in an asymptotic manner.
- Such an analysis is independent of machine type, programming style, etc.

Visualizing Orders of Growth

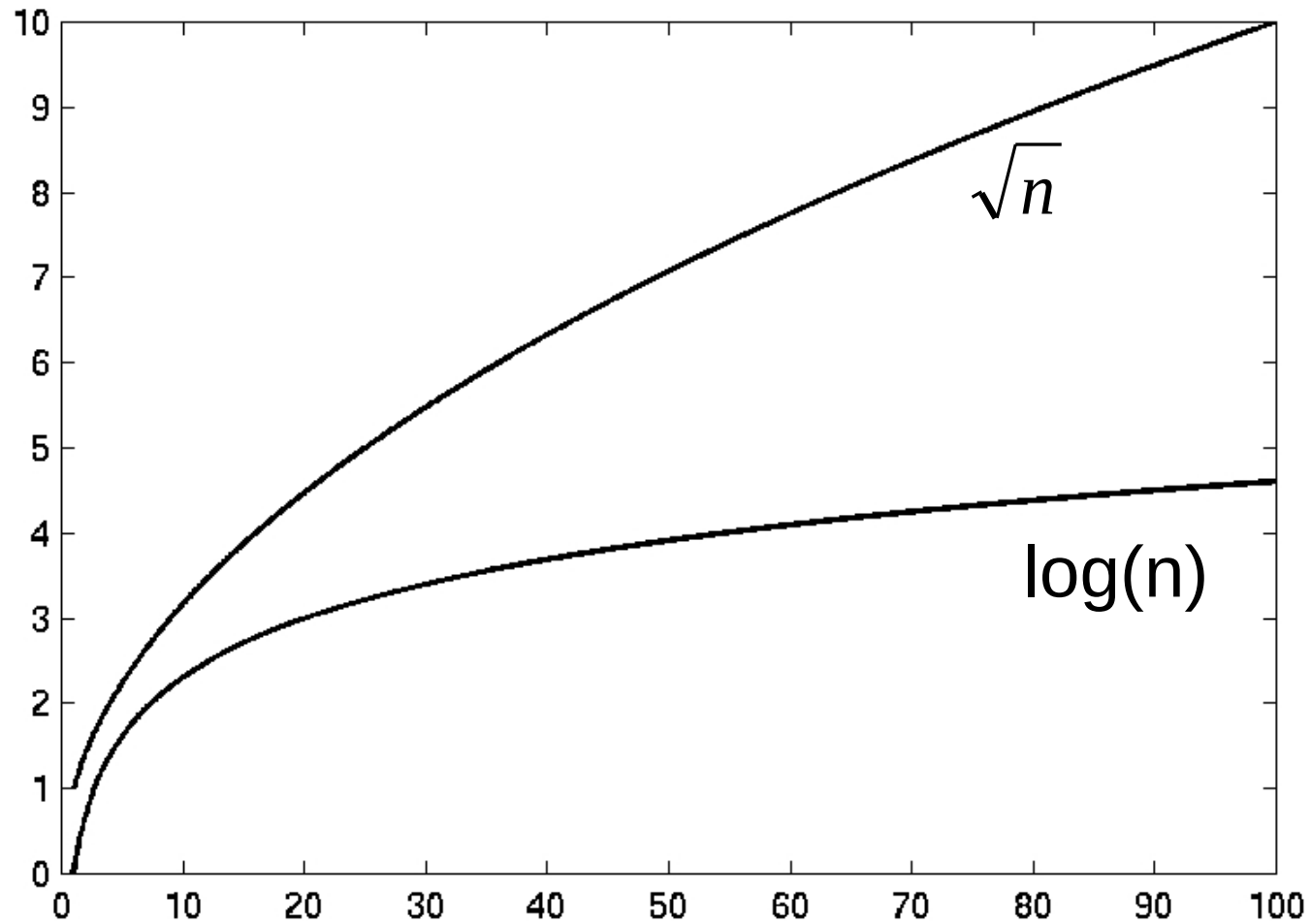
- On a graph, as you go to the right, a faster growing function eventually becomes larger.



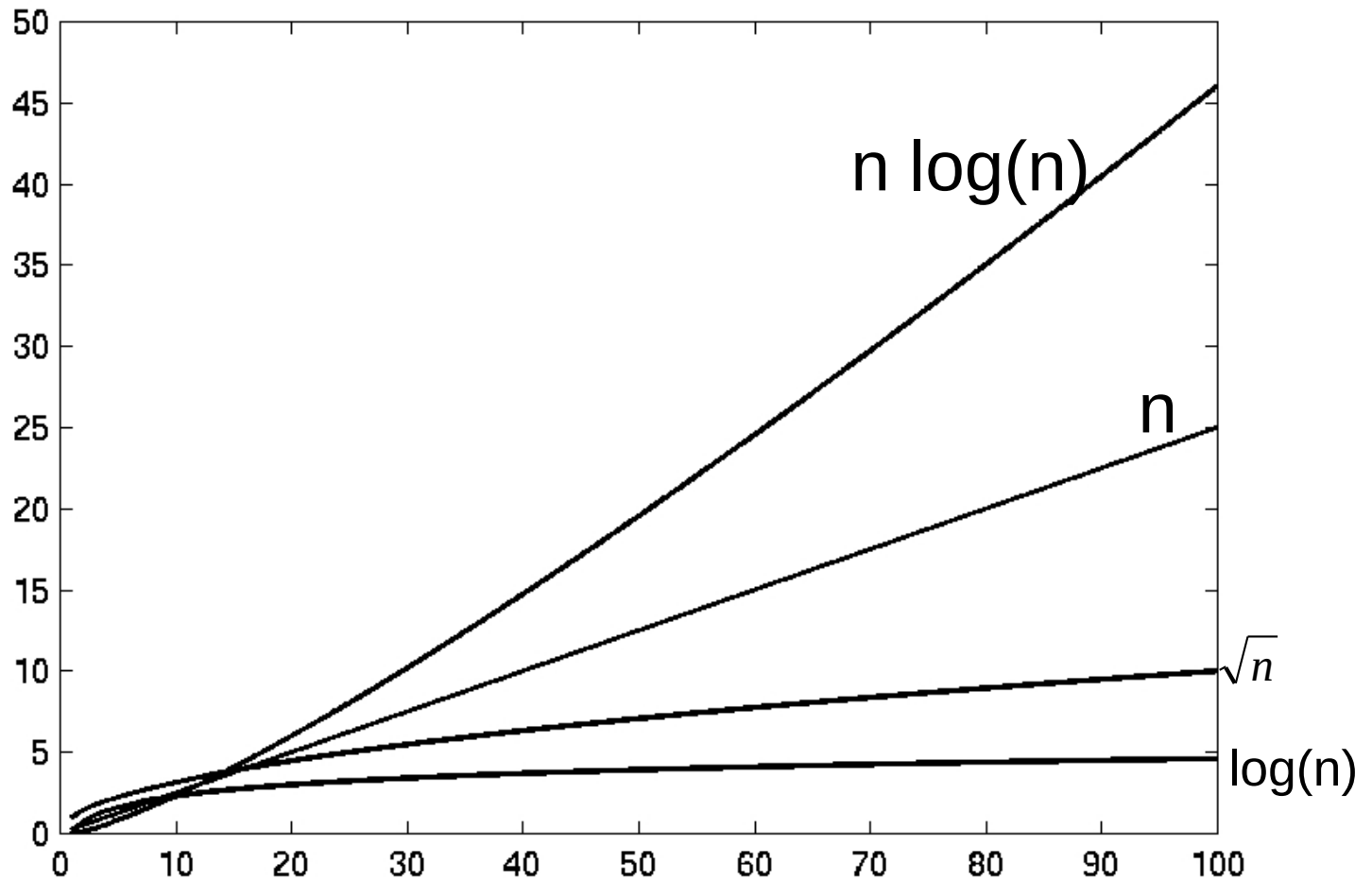
Growth of Functions

n	1	lgn	n	n lgn	n²	n³	2ⁿ
1	1	0.00	1	0	1	1	2
10	1	3.32	10	33	100	1,000	1024
100	1	6.64	100	664	10,000	1,000,000	1.2×10^{30}
1000	1	9.97	1000	9970	1,000,000	10^9	1.1×10^{301}

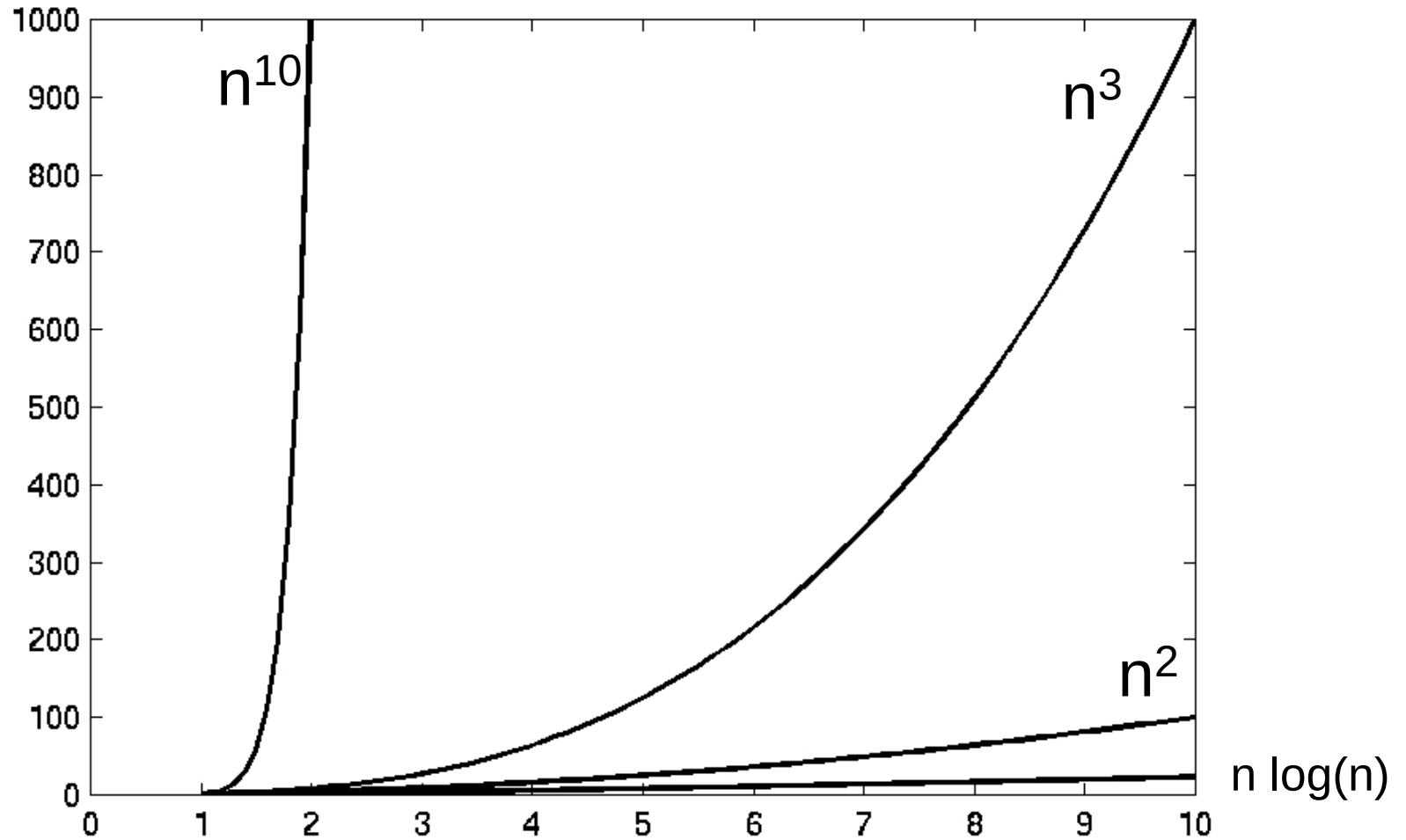
Complexity Graphs



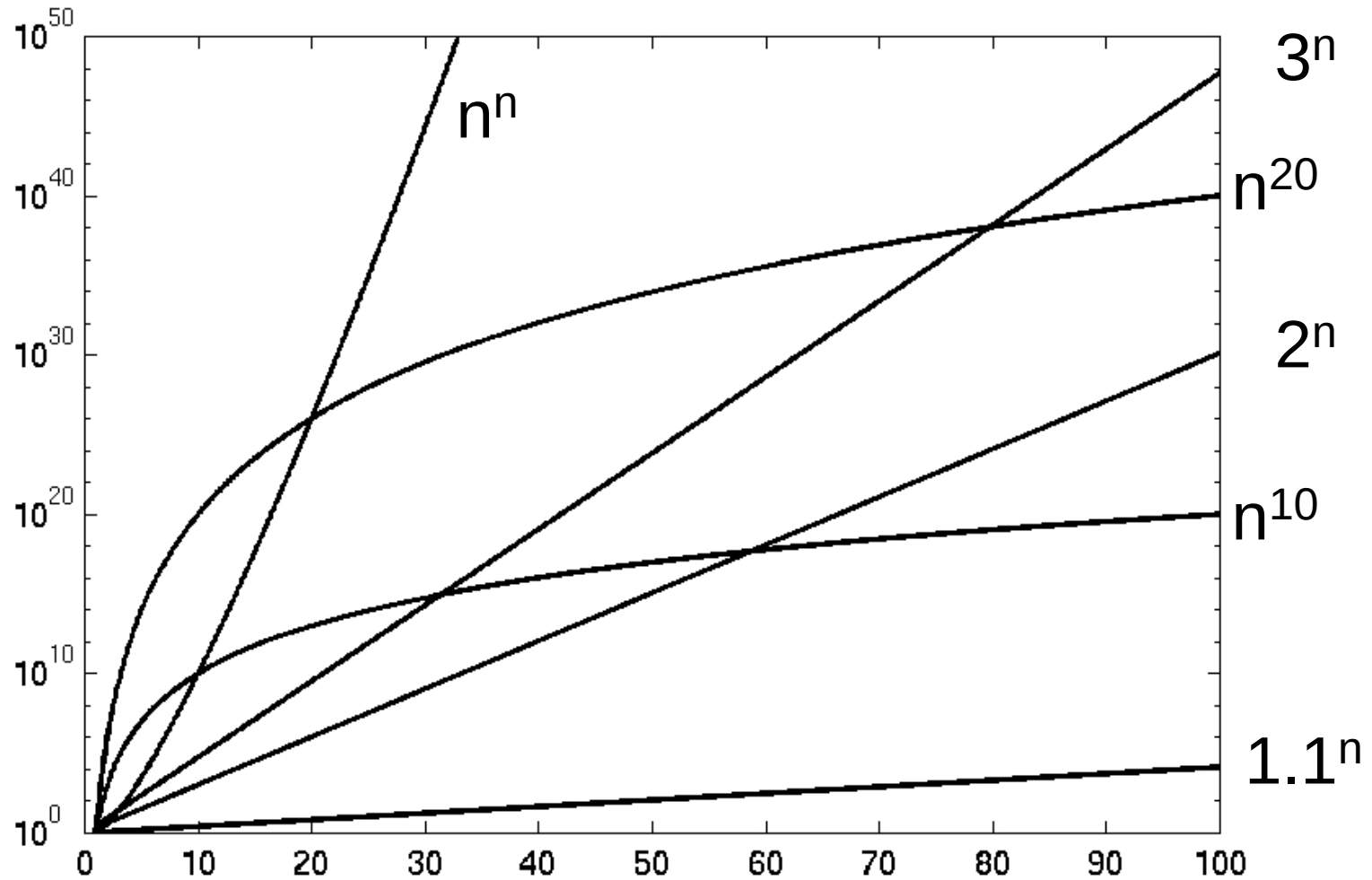
Complexity Graphs



Complexity Graphs



Complexity Graphs (log scale)



Types of Analysis

- Is input size everything that matters?

```
int find_a(char *str)
{
    int i;
    for (i = 0; str[i]; i++)
    {
        if (str[i] == 'a')
            return i;
    }
    return -1;
}
```

- **Time complexity:** $O(n)$
- Consider two inputs: “alibi” and “never”

Types of Analysis

- So how does the running time vary with respect to various input?
- Three scenarios
 - Best case
 - $best_{find_a} = \min_{x \in X_n} runtime_{find_a}$
 - Worst case
 - $worst_{find_a} = \max_{x \in X_n} runtime_{find_a}$
 - Average case
 - $avg_{find_a} = \frac{1}{|X_n|} \sum_{x \in X_n} runtime_{find_a}$

Types of Analysis

- **Worst-case:** (usually done)
 - upper bound on running time
 - maximum running time of algorithm on any input of size n
- **Average-case:** (sometimes done)
 - we take all possible inputs and calculate computing time for all of the inputs
 - sum all the calculated values and divide the sum by total number of inputs
 - we must know (or predict) distribution of cases
- **Best-case:** (bogus)
 - lower bound on running time of an algorithm
 - minimum running time of algorithm on any input of size n

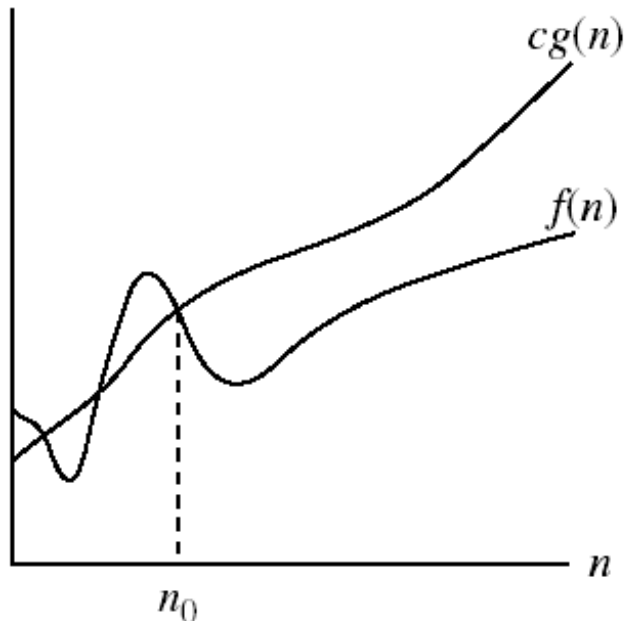
Asymptotic Upper Bound

- Asymptotically bounds the growth of a running time for large values of n to within constant factors above, or in other words, “the running time grows at most this much, but it could grow more slowly”
- A means to compare two algorithms with running times $f(n)$ and $g(n)$, we can establish a **rough measure** that characterizes **how fast each function grows**.
- Compare functions in the limit, that is, **asymptotically!**

Asymptotic Upper Bound

- *O-notation*

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\} .$



$g(n)$ is an *asymptotic upper bound* for $f(n)$.

Asymptotic Upper Bound

- Example:
- Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$.
- For $x > 1$ we have:
 - $x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2$
 - $\Rightarrow x^2 + 2x + 1 \leq 4x^2$
- Therefore, for $C = 4$ and $k = 1$:
- $f(x) \leq Cx^2$ whenever $x > k$.
- $\Rightarrow f(x)$ is $O(x^2)$.

Asymptotic Upper Bound

- We say $f(n) = 30000$ is in the *order of* 1, or **$O(1)$**
 - Growth rate of 30000 is constant, that is, it is not dependent on problem size.
- $f(n) = 30n + 8$ is in the *order of* n , or **$O(n)$**
 - Growth rate of $30n + 8$ is roughly *proportional* to the growth rate of n .
- $f(n) = n^2 + 1$ is in the *order of* n^2 , or **$O(n^2)$**
 - Growth rate of $n^2 + 1$ is roughly proportional to the growth rate of n^2 .
- In general, any $O(n^2)$ function is faster- growing than any $O(n)$ function.
 - For large n , a $O(n^2)$ algorithm runs a lot slower than a $O(n)$ algorithm.

Asymptotic Upper Bound

- Consider the example of buying *elephants* and *goldfish*:

Cost: cost_of_elephants + cost_of_goldfish

Cost $\sim$ cost_of_elephants (approximation)

- Easier way: Discard the low order terms, as well as constants in a function since they are relatively insignificant for **large** n

$$6n + 4 \sim n$$

$$n^4 + 100n^2 + 10n + 50 \sim n^4$$

i.e., we say that $n^4 + 100n^2 + 10n + 50$ and n^4 have the same **rate of growth**

Some Examples

- Determine the time complexity for the following algorithm.

```
count = 0;
```

```
for (i=0; i<10000; i++)
```

```
    count++;
```

Some Examples

- Determine the time complexity for the following algorithm.

```
count = 0;
```

```
for (i=0; i<10000; i++)
```

```
    count++;
```

$O(1)$

Some Examples

- Determine the time complexity for the following algorithm.

```
count = 0;
```

```
for (i=0; i<N; i++)
```

```
    count++;
```

Some Examples

- Determine the time complexity for the following algorithm.

```
count = 0;
```

```
for (i=0; i<N; i++)
```

```
    count++;
```

$O(n)$

Some Examples

- Determine the time complexity for the following algorithm.

```
sum = 0;
```

```
for (i=0; i<N; i++)
```

```
    for (j=0; j<N; j++)
```

```
        sum += arr[i][j];
```

Some Examples

- Determine the time complexity for the following algorithm.

```
sum = 0;
```

```
for (i=0; i<N; i++)
```

$O(n^2)$

```
    for (j=0; j<N; j++)
```

```
        sum += arr[i][j];
```

Some Examples

- Determine the time complexity for the following algorithm.

```
count = 0;
```

```
for (i=1; i<N; i=i*2)
```

```
    count++;
```

Some Examples

- Determine the time complexity for the following algorithm.

```
count = 0;
```

```
for (i=1; i<N; i=i*2)
```

```
    count++;
```

$O(\log n)$

Some Examples

- Determine the time complexity for the following algorithm.

```
char someString[10];  
gets(someString);  
for(i=0; i<strlen(someString); i++)  
    someString[i] -= 32;
```

Some Examples

- Determine the time complexity for the following algorithm.

```
char someString[10];
```

```
gets (someString);
```

$O(n^2)$

```
for (i=0; i<strlen (someString); i++)
```

```
    someString[i] -= 32;
```